Validators

In addition to Pydantic's built-in validation capabilities, you can leverage custom validators at the field and model levels to enforce more complex constraints and ensure the integrity of your data.

Tip

Want to quickly jump to the relevant validator section?

Field validators

- field *after* validators
- field *before* validators
- field *plain* validators
- field *wrap* validators

Model validators

- model *before* validators
- model *after* validators
- model *wrap* validators

Field validators

API Documentation

`pydantic.functional_validators.WrapValidator`
`pydantic.functional_validators.PlainValidator`
`pydantic.functional_validators.BeforeValidator`
`pydantic.functional_validators.AfterValidator`
`pydantic.functional_validators.field_validator`

In its simplest form, a field validator is a callable taking the value to be validated as an argument and **returning the validated value**. The callable can perform checks for specific conditions (see raising validation errors) and make changes to the validated value (coercion or mutation).

Four different types of validators can be used. They can all be defined using the annotated pattern or using the `field_validator()` decorator, applied on a class method:

- ***After* validators**: run after Pydantic's internal validation. They are generally more type safe and thus easier to implement.

Annotated pattern | Decorator

Here is an example of a validator performing a validation check, and returning the value unchanged.

```python
from typing import Annotated

from pydantic import AfterValidator, BaseModel, ValidationError


def is_even(value: int) -> int:
    if value % 2 == 1:
        raise ValueError(f'{value} is not an even number')
    return value


class Model(BaseModel):
    number: Annotated[int, AfterValidator(is_even)]


try:
    Model(number=1)
except ValidationError as err:
    print(err)
    """
    1 validation error for Model
    number
      Value error, 1 is not an even number [type=value_error, input_value=1, input_type=int]
    """
```

1 Note that it is important to return the validated value

Note that it is important to return the validated value.

Example mutating the value

Here is an example of a validator making changes to the validated value (no exception is raised).

Annotated pattern | Decorator

```python
from typing import Annotated

from pydantic import AfterValidator, BaseModel


def double_number(value: int) -> int:
    return value * 2


class Model(BaseModel):
    number: Annotated[int, AfterValidator(double_number)]


print(Model(number=2))
#> number=4
```

- ***Before* validators**: run before Pydantic's internal parsing and validation (e.g. coercion of a `str` to an `int`). These are more flexible than *after* validators, but they also have to deal with the raw input, which in theory could be any arbitrary object. You should also avoid mutating the value directly if you are raising a validation error later in your validator function, as the mutated value may be passed to other validators if using unions.

  The value returned from this callable is then validated against the provided type annotation by Pydantic.

  Annotated pattern | Decorator

  ```python
  from typing import Annotated, Any

  from pydantic import BaseModel, BeforeValidator, ValidationError


  def ensure_list(value: Any) -> Any:  # (1)!
      if not isinstance(value, list):  # (2)!
          return [value]
      else:
          return value


  class Model(BaseModel):
      numbers: Annotated[list[int], BeforeValidator(ensure_list)]


  print(Model(numbers=2))
  #> numbers=[2]
  try:
      Model(numbers='str')
  except ValidationError as err:
      print(err)  # (3)!
      """
      1 validation error for Model
      numbers.0
        Input should be a valid integer, unable to parse string as an integer [type=int_parsing, input_value='str', input_type=st
      """
  ```

  1. Notice the use of `Any` as a type hint for `value`. *Before* validators take the raw input, which can be anything.
  2. Note that you might want to check for other sequence types (such as tuples) that would normally successfully validate against the `list` type. *Before* validators give you more flexibility, but you have to account for every possible case.
  3. Pydantic still performs validation against the `int` type, no matter if our `ensure_list` validator did operations on the original input type.

- ***Plain* validators**: act similarly to *before* validators but they **terminate validation immediately** after returning, so no further validators are called and Pydantic does not do any of its internal validation against the field type.

  Annotated pattern | Decorator

  ```python
  from typing import Annotated, Any

  from pydantic import BaseModel, PlainValidator


  def val_number(value: Any) -> Any:
      if isinstance(value, int):
          return value * 2
      else:
  ```

```
        return value


class Model(BaseModel):
    number: Annotated[int, PlainValidator(val_number)]


print(Model(number=4))
#> number=8
print(Model(number='invalid'))
#> number='invalid'
```

1. Although `'invalid'` shouldn't validate against the `int` type, Pydantic accepts the input.

- ***Wrap* validators**: are the most flexible of all. You can run code before or after Pydantic and other validators process the input, or you can terminate validation immediately, either by returning the value early or by raising an error.

  Such validators must be defined with a **mandatory** extra *handler* parameter: a callable taking the value to be validated as an argument. Internally, this handler will delegate validation of the value to Pydantic. You are free to wrap the call to the handler in a `try..except` block, or not call it at all.

  Annotated pattern | Decorator

```
from typing import Any

from typing import Annotated

from pydantic import BaseModel, Field, ValidationError, ValidatorFunctionWrapHandler, WrapValidator


def truncate(value: Any, handler: ValidatorFunctionWrapHandler) -> str:
    try:
        return handler(value)
    except ValidationError as err:
        if err.errors()[0]['type'] == 'string_too_long':
            return handler(value[:5])
        else:
            raise


class Model(BaseModel):
    my_string: Annotated[str, Field(max_length=5), WrapValidator(truncate)]


print(Model(my_string='abcde'))
#> my_string='abcde'
print(Model(my_string='abcdef'))
#> my_string='abcde'
```

Validation of default values

As mentioned in the fields documentation, default values of fields are *not* validated unless configured to do so, and thus custom validators will not be applied as well.

Which validator pattern to use

While both approaches can achieve the same thing, each pattern provides different benefits.

Using the annotated pattern

One of the key benefits of using the annotated pattern is to make validators reusable:

```
from typing import Annotated

from pydantic import AfterValidator, BaseModel


def is_even(value: int) -> int:
    if value % 2 == 1:
        raise ValueError(f'{value} is not an even number')
    return value


EvenNumber = Annotated[int, AfterValidator(is_even)]


class Model1(BaseModel):
    my_number: EvenNumber
```

```python
class Model2(BaseModel):
    other_number: Annotated[EvenNumber, AfterValidator(lambda v: v + 2)]


class Model3(BaseModel):
    list_of_even_numbers: list[EvenNumber]
```

1. As mentioned in the annotated pattern documentation, we can also make use of validators for specific parts of the annotation (in this case, validation is applied for list items, but not the whole list).

It is also easier to understand which validators are applied to a type, by just looking at the field annotation.

Using the decorator pattern

One of the key benefits of using the `field_validator()` decorator is to apply the function to multiple fields:

```python
from pydantic import BaseModel, field_validator


class Model(BaseModel):
    f1: str
    f2: str

    @field_validator('f1', 'f2', mode='before')
    @classmethod
    def capitalize(cls, value: str) -> str:
        return value.capitalize()
```

Here are a couple additional notes about the decorator usage:

- If you want the validator to apply to all fields (including the ones defined in subclasses), you can pass `'*'` as the field name argument.
- By default, the decorator will ensure the provided field name(s) are defined on the model. If you want to disable this check during class creation, you can do so by passing `False` to the `check_fields` argument. This is useful when the field validator is defined on a base class, and the field is expected to exist on subclasses.

Model validators ¶

API **API Documentation**

`pydantic.functional_validators.model_validator`

Validation can also be performed on the entire model's data using the `model_validator()` decorator.

Three different types of model validators can be used:

- ***After* validators**: run after the whole model has been validated. As such, they are defined as *instance* methods and can be seen as post-initialization hooks. Important note: the validated instance should be returned.

```python
from typing_extensions import Self

from pydantic import BaseModel, model_validator


class UserModel(BaseModel):
    username: str
    password: str
    password_repeat: str

    @model_validator(mode='after')
    def check_passwords_match(self) -> Self:
        if self.password != self.password_repeat:
            raise ValueError('Passwords do not match')
        return self
```

- ***Before* validators**: are run before the model is instantiated. These are more flexible than *after* validators, but they also have to deal with the raw input, which in theory could be any arbitrary object. You should also avoid mutating the value directly if you are raising a validation error later in your validator function, as the mutated value may be passed to other validators if using unions.

```python
from typing import Any

from pydantic import BaseModel, model_validator


class UserModel(BaseModel):
```

```
    username: str

    @model_validator(mode='before')
    @classmethod
    def check_card_number_not_present(cls, data: Any) -> Any:
        if isinstance(data, dict):
            if 'card_number' in data:
                raise ValueError("'card_number' should not be included")
        return data
```

1. Notice the use of `Any` as a type hint for `data`. *Before* validators take the raw input, which can be anything.
2. Most of the time, the input data will be a dictionary (e.g. when calling `UserModel(username='...')`). However, this is not always the case. For instance, if the `from_attributes` configuration value is set, you might receive an arbitrary class instance for the `data` argument.

- ***Wrap* validators**: are the most flexible of all. You can run code before or after Pydantic and other validators process the input data, or you can terminate validation immediately, either by returning the data early or by raising an error.

```
import logging
from typing import Any

from typing_extensions import Self

from pydantic import BaseModel, ModelWrapValidatorHandler, ValidationError, model_validator


class UserModel(BaseModel):
    username: str

    @model_validator(mode='wrap')
    @classmethod
    def log_failed_validation(cls, data: Any, handler: ModelWrapValidatorHandler[Self]) -> Self:
        try:
            return handler(data)
        except ValidationError:
            logging.error('Model %s failed to validate with data %s', cls, data)
            raise
```

On inheritance

A model validator defined in a base class will be called during the validation of a subclass instance.

Overriding a model validator in a subclass will override the base class' validator, and thus only the subclass' version of said validator will be called.

Raising validation errors

To raise a validation error, three types of exceptions can be used:

- `ValueError`: this is the most common exception raised inside validators.
- `AssertionError`: using the assert statement also works, but be aware that these statements are skipped when Python is run with the -O optimization flag.
- `PydanticCustomError`: a bit more verbose, but provides extra flexibility:

```
from pydantic_core import PydanticCustomError

from pydantic import BaseModel, ValidationError, field_validator


class Model(BaseModel):
    x: int

    @field_validator('x', mode='after')
    @classmethod
    def validate_x(cls, v: int) -> int:
        if v % 42 == 0:
            raise PydanticCustomError(
                'the_answer_error',
                '{number} is the answer!',
                {'number': v},
            )
        return v


try:
    Model(x=42 * 2)
except ValidationError as e:
    print(e)
```

```
print(e)
"""
1 validation error for Model
x

  84 is the answer! [type=the_answer_error, input_value=84, input_type=int]
"""
```

Validation info

Both the field and model validators callables (in all modes) can optionally take an extra `ValidationInfo` argument, providing useful extra information, such as:

- already validated data
- user defined context
- the current validation mode: either `'python'`, `'json'` or `'strings'` (see the `mode` property)
- the current field name, if using a field validator (see the `field_name` property).

Validation data

For field validators, the already validated data can be accessed using the `data` property. Here is an example than can be used as an alternative to the *after* model validator example:

```
from pydantic import BaseModel, ValidationInfo, field_validator


class UserModel(BaseModel):
    password: str
    password_repeat: str
    username: str

    @field_validator('password_repeat', mode='after')
    @classmethod
    def check_passwords_match(cls, value: str, info: ValidationInfo) -> str:
        if value != info.data['password']:
            raise ValueError('Passwords do not match')
        return value
```

Warning

As validation is performed in the order fields are defined, you have to make sure you are not accessing a field that hasn't been validated yet. In the code above, for example, the `username` validated value is not available yet, as it is defined *after* `password_repeat`.

The `data` property is `None` for model validators.

Validation context

You can pass a context object to the validation methods, which can be accessed inside the validator functions using the `context` property:

```
from pydantic import BaseModel, ValidationInfo, field_validator


class Model(BaseModel):
    text: str

    @field_validator('text', mode='after')
    @classmethod
    def remove_stopwords(cls, v: str, info: ValidationInfo) -> str:
        if isinstance(info.context, dict):
            stopwords = info.context.get('stopwords', set())
            v = ' '.join(w for w in v.split() if w.lower() not in stopwords)
        return v


data = {'text': 'This is an example document'}
print(Model.model_validate(data))  # no context
#> text='This is an example document'
print(Model.model_validate(data, context={'stopwords': ['this', 'is', 'an']}))
#> text='example document'
```

Similarly, you can use a context for serialization.

Providing context when directly instantiating a model

It is currently not possible to provide a context when directly instantiating a model (i.e. when calling `Model(...)`). You can work around this through the use of a `ContextVar` and a custom `__init__` method:

```python
from __future__ import annotations

from collections.abc import Generator
from contextlib import contextmanager
from contextvars import ContextVar
from typing import Any

from pydantic import BaseModel, ValidationInfo, field_validator

_init_context_var = ContextVar('_init_context_var', default=None)


@contextmanager
def init_context(value: dict[str, Any]) -> Generator[None]:
    token = _init_context_var.set(value)
    try:
        yield
    finally:
        _init_context_var.reset(token)


class Model(BaseModel):
    my_number: int

    def __init__(self, /, **data: Any) -> None:
        self.__pydantic_validator__.validate_python(
            data,
            self_instance=self,
            context=_init_context_var.get(),
        )

    @field_validator('my_number')
    @classmethod
    def multiply_with_context(cls, value: int, info: ValidationInfo) -> int:
        if isinstance(info.context, dict):
            multiplier = info.context.get('multiplier', 1)
            value = value * multiplier
        return value


print(Model(my_number=2))
#> my_number=2

with init_context({'multiplier': 3}):
    print(Model(my_number=2))
    #> my_number=6

print(Model(my_number=2))
#> my_number=2
```

Ordering of validators

When using the annotated pattern, the order in which validators are applied is defined as follows: *before* and *wrap* validators are run from right to left, and *after* validators are then run from left to right:

```python
from pydantic import AfterValidator, BaseModel, BeforeValidator, WrapValidator


class Model(BaseModel):
    name: Annotated[
        str,
        AfterValidator(runs_3rd),
        AfterValidator(runs_4th),
        BeforeValidator(runs_2nd),
        WrapValidator(runs_1st),
    ]
```

Internally, validators defined using the decorator are converted to their annotated form counterpart and added last after the existing metadata for the field. This means that the same ordering logic applies.

Special types

Pydantic provides a few special utilities that can be used to customize validation.

- `InstanceOf` can be used to validate that a value is an instance of a given class.

  ```python
  from pydantic import BaseModel, InstanceOf, ValidationError
  ```

```python
class Fruit:
    def __repr__(self):
        return self.__class__.__name__


class Banana(Fruit): ...


class Apple(Fruit): ...


class Basket(BaseModel):
    fruits: list[InstanceOf[Fruit]]


print(Basket(fruits=[Banana(), Apple()]))
#> fruits=[Banana, Apple]
try:
    Basket(fruits=[Banana(), 'Apple'])
except ValidationError as e:
    print(e)
    """
    1 validation error for Basket
    fruits.1
      Input should be an instance of Fruit [type=is_instance_of, input_value='Apple', input_type=str]
    """
```

- `SkipValidation` can be used to skip validation on a field.

```python
from pydantic import BaseModel, SkipValidation


class Model(BaseModel):
    names: list[SkipValidation[str]]


m = Model(names=['foo', 'bar'])
print(m)
#> names=['foo', 'bar']

m = Model(names=['foo', 123])
print(m)
#> names=['foo', 123]
```

(1) Note that the validation of the second item is skipped. If it has the wrong type it will emit a warning during serialization.

- `ValidateAs` can be used to validate an custom type from a type natively supported by Pydantic. This is particularly useful when using custom types with multiple fields.

```python
from typing import Annotated

from pydantic import BaseModel, TypeAdapter, ValidateAs

class MyCls:
    def __init__(self, a: int) -> None:
        self.a = a

    def __repr__(self) -> str:
        return f"MyCls(a={self.a})"

class ValModel(BaseModel):
    a: int


ta = TypeAdapter(
    Annotated[MyCls, ValidateAs(ValModel, lambda v: MyCls(a=v.a))]
)

print(ta.validate_python({'a': 1}))
#> MyCls(a=1)
```

- `PydanticUseDefault` can be used to notify Pydantic that the default value should be used.

```python
from typing import Annotated, Any

from pydantic_core import PydanticUseDefault

from pydantic import BaseModel, BeforeValidator
```

```python
def default_if_none(value: Any) -> Any:
    if value is None:
        raise PydanticUseDefault()

    return value


class Model(BaseModel):
    name: Annotated[str, BeforeValidator(default_if_none)] = 'default_name'


print(Model(name=None))
#> name='default_name'
```

JSON Schema and field validators

When using *before*, *plain* or *wrap* field validators, the accepted input type may be different from the field annotation.

Consider the following example:

```python
from typing import Any

from pydantic import BaseModel, field_validator


class Model(BaseModel):
    value: str

    @field_validator('value', mode='before')
    @classmethod
    def cast_ints(cls, value: Any) -> Any:
        if isinstance(value, int):
            return str(value)
        else:
            return value


print(Model(value='a'))
#> value='a'
print(Model(value=1))
#> value='1'
```

While the type hint for `value` is `str`, the `cast_ints` validator also allows integers. To specify the correct input type, the `json_schema_input_type` argument can be provided:

```python
from typing import Any, Union

from pydantic import BaseModel, field_validator


class Model(BaseModel):
    value: str

    @field_validator(
        'value', mode='before', json_schema_input_type=Union[int, str]
    )
    @classmethod
    def cast_ints(cls, value: Any) -> Any:
        if isinstance(value, int):
            return str(value)
        else:
            return value


print(Model.model_json_schema()['properties']['value'])
#> {'anyOf': [{'type': 'integer'}, {'type': 'string'}], 'title': 'Value'}
```

As a convenience, Pydantic will use the field type if the argument is not provided (unless you are using a *plain* validator, in which case `json_schema_input_type` defaults to `Any` as the field type is completely discarded).